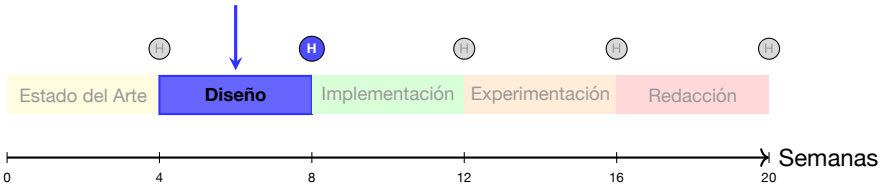
Documento hito #6 (de 6) de mes 2



Mini-Prototipo de Evaluación Paralela con Spark RDD

TFM: Selección Automática de Características en Algoritmos de Clasificación Mediante Algoritmos Genéticos y Métodos de Envoltorio con Apache Spark RDD

Notebook: `notebooks/01_parallel_fitness_prototype.ipynb`

---

1. Objetivo

Validar experimentalmente que la arquitectura de paralelización diseñada en el documento de arquitectura Spark RDD funciona correctamente antes de implementar el GA completo. En concreto, se verifica que:

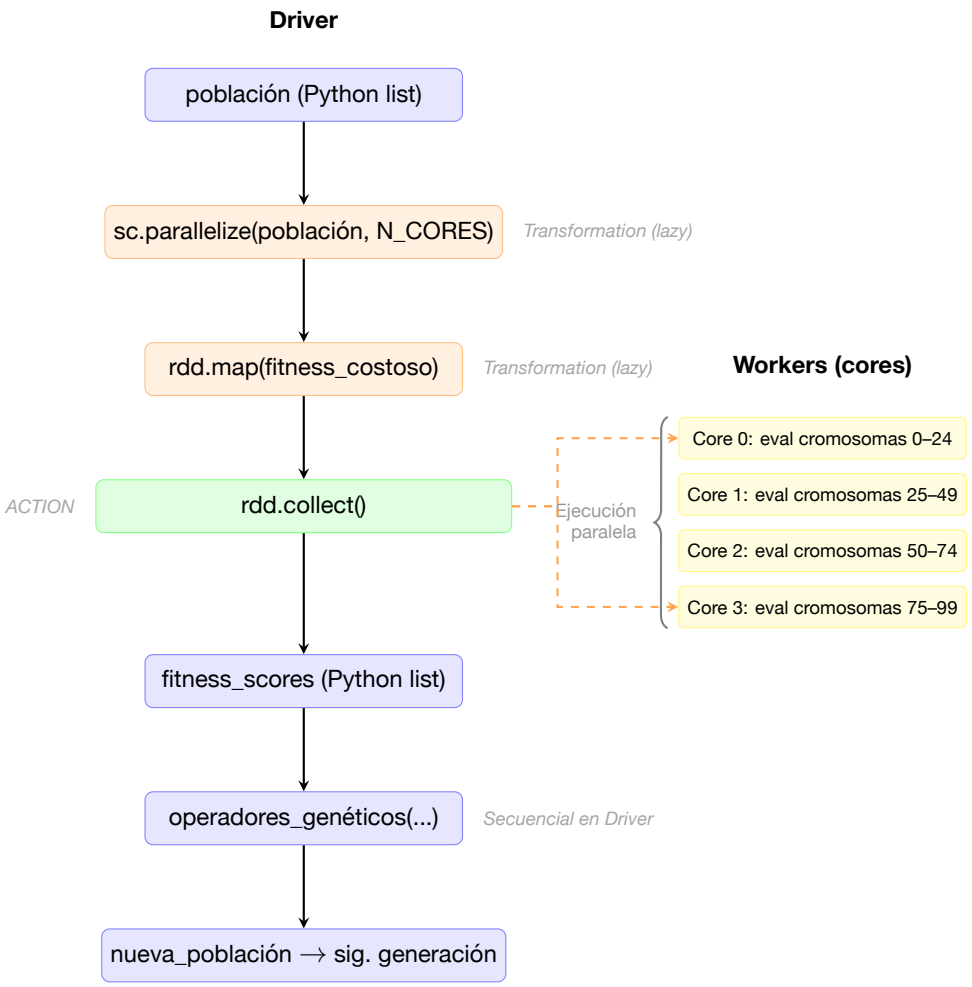
1. Una población de cromosomas puede distribuirse como RDD y evaluarse en paralelo con `rdd.map()`.
2. El paralelismo produce un speedup real medible frente a la ejecución secuencial.
3. Los resultados paralelos son idénticos a los secuenciales (correctitud).
4. La ejecución es observable en la Spark Web UI (`http://localhost:4040`).
5. El patrón `parallelize → map → collect` funciona dentro de un bucle generacional del GA.

2. Relación con la Arquitectura del Proyecto

Este prototipo implementa de forma simplificada el patrón descrito en los documentos de diseño del TFM:

- **Documento de diseño del GA (`diseño_GA.md`)**: Define el cromosoma binario de longitud D, la función de fitness multiobjetivo $\text{Fitness}(\mathbf{c}) = \alpha \cdot \text{Accuracy}(\mathbf{c}) - \beta \cdot |S|/D$, y el ciclo evolutivo con evaluación paralela via `RDD.map()`.
- **Documento de arquitectura Spark (`arquitectura_spark_rdd_ga.md`)**: Establece la separación Driver/Workers, donde la evaluación de fitness se ejecuta en los Workers (paralelizable, patrón *embarrassingly parallel*) y los operadores genéticos se ejecutan secuencialmente en el Driver.

El prototipo replica exactamente este patrón:



3. Descripción del Código

3.1 Población ficticia

Se genera una población de $P = 100$ individuos con cromosomas binarios de longitud $D = 30$ (equivalente al dataset WDBC del TFM). Cada gen se inicializa con probabilidad $p_{init} = 0.5$:

```
poblacion = [list(np.random.binomial(1, P_INIT, D)) for _ in range(P)]
```

Esto produce cromosomas como `[1, 0, 1, 1, 0, 0, 1, 0, ...]` donde cada posición indica si la característica correspondiente está incluida en el subconjunto candidato.

3.2 Función fitness dummy

Se implementan dos variantes:

- `fitness_rapido`: Calcula un pseudo-accuracy determinista a partir de un hash del cromosoma. El hash se usa como semilla de `numpy.RandomState`, produciendo un valor en `[0.5, 1.0]` —rango realista para un clasificador mejor que el azar— de forma que el fitness nunca toma valores negativos. No introduce retardo artificial; sirve para validar correctitud.
- `fitness_costoso`: Añade `time.sleep(sleep_ms)` para simular el coste real de entrenar un clasificador con 5-fold cross-validation. Con `sleep_ms=50` se simula un coste de 50 ms por evaluación, representativo de Naive Bayes sobre WDBC. Este es el test clave para demostrar el speedup.

Ambas funciones siguen la fórmula de fitness del diseño con $\alpha = 0.9$ y $\beta = 0.1$:

```
seed     = hash(tuple(cromosoma)) % (2**32)
rng      = np.random.RandomState(seed)
accuracy = 0.5 + 0.5 * rng.random()          # en [0.5, 1.0]
fitness  = ALPHA * accuracy - BETA * (genes_activos / D)
```

El rango teórico resultante es $[0.35,\ 0.897]$, sin valores negativos.

La función es determinista: el mismo cromosoma produce siempre el mismo fitness, lo que garantiza que la validación `seq == paralelo` sea válida independientemente del orden de evaluación en los Workers.

El caso especial de cromosoma nulo ($|S| = 0$) retorna fitness $= 0$.

3.3 Evaluación secuencial (baseline)

Evalúa los 100 individuos con un bucle Python estándar:

```
fitness_seq = [fitness_costoso(ind, sleep_ms=50) for ind in poblacion]
```

Con $P = 100$ y sleep $= 50$ms, el tiempo esperado es ~ 5.0s.

3.4 Evaluación paralela (Spark RDD)

Distribuye la población como RDD con tantas particiones como cores disponibles y evalúa en paralelo:

```
rdd_poblacion = sc.parallelize(poblacion, numSlices=NUM_CORES)
fitness_par = rdd_poblacion.map(lambda ind: fitness_costoso(ind, sleep_ms=50)).collect()
```

Este es el patrón `parallelize → map → collect` descrito en la arquitectura. Según el documento de arquitectura, con `local[N]` se procesan N individuos simultáneamente, completando la población en `ceil(P/N)` rondas.

Tiempo esperado con 4 cores: $\sim 100 \times 50\text{ms}/4 = 1.25$s (speedup teórico ≈ 4x).

3.5 Validación de correctitud

Se compara elemento a elemento que los resultados secuenciales y paralelos coinciden:

```
resultados_iguales = all(abs(s - p) < 1e-9 for s, p in zip(fitness_seq, fitness_par))
```

Dado que la función fitness es determinista y no depende de estado compartido, los resultados deben ser idénticos. Esto confirma que la serialización/deserialización de Spark no altera los cálculos.

3.6 Experimento de escalabilidad

Se varía el número de particiones del RDD (1, 2, 4, N, 2N) para observar cómo escala el speedup. Se generan dos gráficos:

1. **Tiempo vs. Particiones**: Muestra la reducción de tiempo al aumentar particiones, con la línea baseline secuencial como referencia.
2. **Speedup vs. Particiones**: Compara el speedup obtenido con el speedup ideal (lineal).

El speedup sub-lineal es esperado por el overhead de serialización, scheduling de tareas y recolección de resultados por parte del Driver.

3.7 Simulación de generaciones del GA

Se ejecutan 5 generaciones completas del ciclo evolutivo para validar el patrón en un bucle:

1. **Evaluación paralela** (Spark): `sc.parallelize → map(fitness) → collect()`

2. **Selección** (Driver): Torneo binario de tamaño 2
3. **Cruce** (Driver): Un punto con $p_{cross} = 0.8$
4. **Mutación** (Driver): Flip de bit con $p_{mut} = 0.2, p_{mut_bit} = 1/D$
5. **Elitismo** (Driver): Top 2 individuos pasan directamente

Los operadores genéticos se implementan siguiendo las especificaciones del documento de diseño del GA y se ejecutan secuencialmente en el Driver, como establece la arquitectura.

4. Métricas Capturadas

Métrica	Descripción
Tiempo secuencial	Baseline: tiempo total de evaluar P individuos con Python puro
Tiempo paralelo	Tiempo con Spark RDD (`map` + `collect`)
Speedup	t_{seq}/t_{par} — factor de aceleración
Eficiencia	speedup$/N_{cores} \times 100\%$
Correctitud	`fitness_seq[i] == fitness_par[i]` para todo i

5. Qué Observar en la Spark Web UI

Al ejecutar el notebook, acceder a `http://localhost:4040` para verificar:

Pestaña	Qué observar
Jobs	Un job por cada `collect()`. En el experimento de escalabilidad aparecerán varios jobs consecutivos. En la simulación de generaciones, 5 jobs (uno por generación).
Stages	Cada job tiene un stage con tantas tareas como particiones. Verificar que las tareas se ejecutan en paralelo (columna de tiempos solapados).
Executors	Verificar que los cores están activos durante la evaluación. La columna “Active Tasks” debe mostrar N_{cores} tareas simultáneas.
Environment	Confirmar la configuración: `spark.default.parallelism`, `spark.driver.memory`, etc.

6. Criterios de Validación

El prototipo se considera exitoso si cumple todos los siguientes criterios:

#	Criterio	Umbral
1	Población distribuida como RDD	Sin errores
2	Evaluación paralela con `rdd.map()`	Sin errores
3	Correctitud (seq == paralelo)	100% coincidencia
4	Speedup significativo	> 1.5x
5	Ciclo evolutivo completo	5 generaciones sin error
6	Observable en Web UI	Jobs visibles en localhost:4040

7. Limitaciones del Prototipo

- **Fitness simulado**: Se usa `time.sleep` en lugar de un clasificador real. Esto aísla el efecto del paralelismo pero no captura el overhead de scikit-learn ni el broadcasting de datos.
- **Datos no distribuidos**: El dataset de entrenamiento no se incluye; en la implementación real se usarán *broadcast variables* para enviar los datos a los Workers.
- **Escala local**: `local[*]` ejecuta todo en una JVM. Un cluster real tendría overhead de red adicional, pero también más recursos.
- **Pocas generaciones**: 5 generaciones bastan para validar el patrón, pero no para observar convergencia del GA.

8. Siguiente Paso

Una vez validado el paralelismo con este prototipo, el siguiente paso es sustituir la función fitness dummy por la evaluación real con Naive Bayes gaussiano y validación cruzada de 5 pliegues, incorporando:

1. Broadcasting del dataset de entrenamiento a los Workers.
2. Integración con DEAP para los operadores genéticos.
3. Criterio de parada dual (generaciones máximas + convergencia).
4. Evaluación sobre el dataset WDBC ($D = 30$) como primer caso de prueba.